

Deep RL for Automated Stock Trading

Weekly Progress Report — Week 9

Rutwik Shrikant Zhu Yufan Fei Xin

Semester 2, 2025/26

1 Summary

This week has focused on understanding the stock dataset, cleaning and transforming the raw time series, and implementing the first version of the trading environment. The main goal at this stage has been to establish a reliable data pipeline and a Gym-style environment before introducing any reinforcement learning agent.

2 Data Analysis and Cleaning

We have used `yfinance` to download daily OHLCV data for DJIA stocks from 2009–2024. The main data work this week has centred on checking data completeness, aligning price and volume fields, and handling the warm-up periods introduced by rolling-window indicators. For each ticker, we have constructed a compact feature set consisting of daily return, short- and medium-term moving-average deviations, RSI-14, the MACD signal histogram, Bollinger Band width, normalised volume, ATR, and normalised OBV. This gives us a mixture of momentum, trend, volatility, and volume information without depending directly on raw price levels alone.

After feature engineering, we have removed rows with missing values caused by indicator warm-up windows. We have then applied a chronological train/validation/test split with a 70%/15%/15% ratio and normalised the features using **training-set statistics only** to avoid look-ahead bias. By the end of the week, the preprocessing pipeline has been stable enough to support repeatable experiments on a clean dataset.

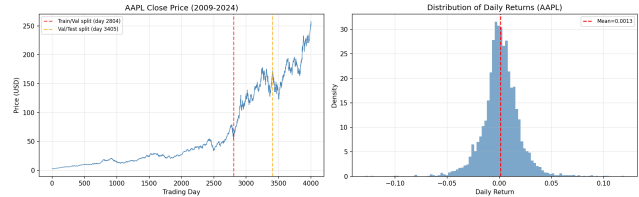


Figure 1: Initial exploration of AAPL: price trajectory and daily return distribution.

3 Environment Implementation

We have implemented the first version of `TradingEnv`, a Gym-style single-asset trading environment. The state vector contains the engineered market features together with the agent’s current position. The environment supports three discrete actions and maintains internal histories for positions, rewards, trades, and portfolio value.

We have also defined the core reward structure as:

$$r_t = \text{PnL}_t - \text{transaction cost}_t + \text{risk penalty}_t \quad (1)$$

This first version already includes transaction costs and portfolio tracking, which is important for discouraging unrealistic behaviour during later training. At this point, the environment is not yet focused on peak performance, but it captures the basic mechanics needed for agent development next week.

4 Next Steps

Next week we plan to build the DQN and DDQN agents from scratch in PyTorch, connect them to the environment, and complete the first end-to-end training and evaluation pipeline.